AGENT SUBSTRATE DAY

kubectl is lying to you

What a live visualizer taught me about Agent Substrate

Mike Moore • Solo.io



ATECon • Monday, November 9, 2026 • Sky SLC

Two quiet pods. Eighteen agents.

kind-substrate-day

```
$ kubectl get pods -n kagent -l ate.dev/worker-pool
NAME                              READY   STATUS    RESTARTS   AGE
kagent-default-7f5fcb797f-zc8sc   1/1     Running   0          12m
kagent-oncall-97ccb757c-bnvp5     1/1     Running   0          20m

$ kubectl get agents -n kagent --no-headers | wc -l
      18

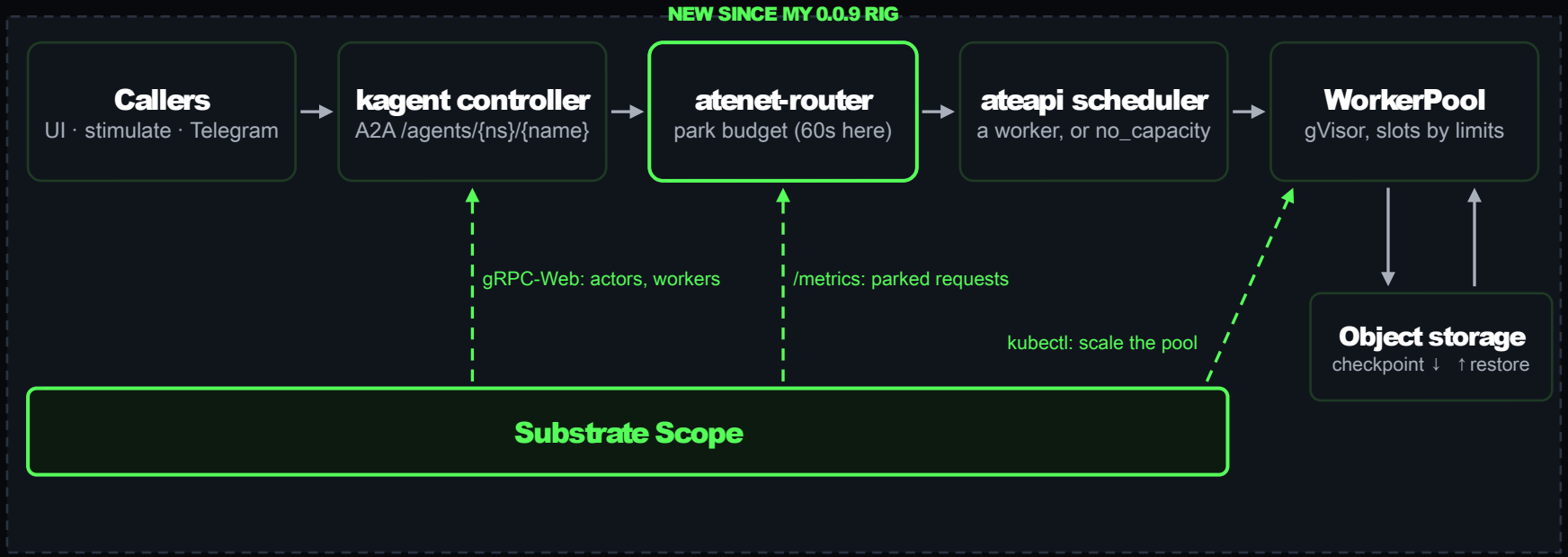
# Right now those pods hold zero of them. All 18 are zstd snapshots
# in object storage, reserving no CPU or memory until someone says hi.
```

I'm the guy who needs the hard stuff drawn in crayon. So I picked up the purple one.

Harold and the Purple Crayon, 1955

(GIF in the live talk)

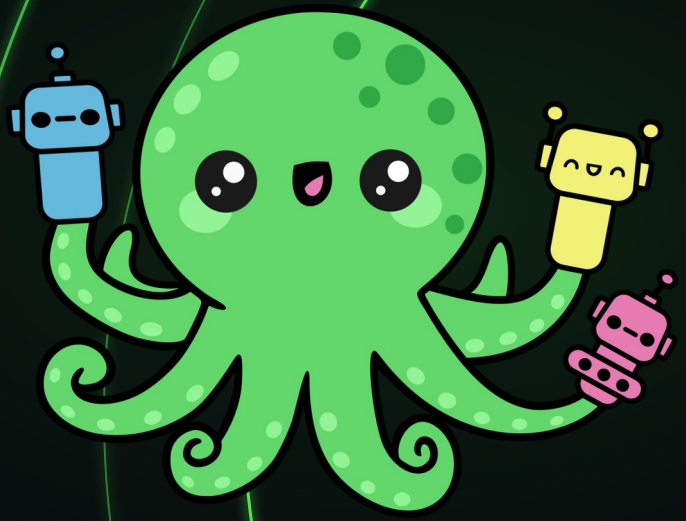
One restore, and where Scope looks



No single API shows the whole lifecycle, so Scope reads three: kagent, the router, and the WorkerPool.

LIVE DEMO

What could possibly go wrong?



Five minutes on the board: localhost:8123

1. kubectl says two pods. The board says eighteen agents.
2. Talk to one: restore, answer, checkpoint.
3. SURGE: 13 chats onto 2 slots. The router parks the rest.
4. AUTOSCALE up, then back down on its own.
5. Text sre-oncall from my phone.

3.6x

less reserved CPU than 18 always-on pods

6.75x less memory.

Worker pods at rest on kagent 1.0.0-rc3 + Substrate 1.0.0: 2 × 250m/512Mi vs 18 × 100m/384Mi (kagent 0.10.x pod defaults). Substrate's control plane adds a fixed ~0.5 CPU / 1.5Gi, so the gap widens as agents grow.

Read the workers, not the actors

kagent.api.v1.SystemService

```
ListSubstrateWorkers   →     2 workers
ListSubstrateActors    → 1,017 actors  (1,008 SUSPENDED, 9 CRASHED)

# One day of rehearsal, 18 agents. Every finished session leaves
# a Suspended actor behind.
# 0.0.9: my board read the actors list; it grew without bound and
#        started returning partial results. Running sessions vanished.
# 1.0:   actors page at 100, and workers no longer name their actor.
truth = actor.status.worker_assignment ⋈ workers
```

Your HPA is staring at the wrong number

What CPU and memory see

- Busy sessions: tens of mCPU, flat memory
- A saturated pool looks idle
- An HPA scales down at peak load

What the pool needs

- Up: busy + waiting, in one jump
- Down: the peak of a 30s window
- Now: parking_active, no_capacity

Three ways I fooled myself

01

Scale-out hurt

A serial model backend turned scale-out into a longer convoy

02

Ghost actors

Interrupted sessions pinned workers while the pool looked idle

03

Usage flatters

Idle pods use almost no CPU but reserve it forever

> **Interrupted sessions left ghost actors pinning workers while the pool looked idle.**

This made me sad. (It's fixable.)



Nobody expects a 1.0 days before their talk.



Built on rc2 Nov 4. Substrate 1.0 went GA Nov 5, kagent rc3 Nov 6. This rig moved to 1.0 on Friday.

1.0 moved. My board broke quietly.

kagent 1.0.0-rc3 · Substrate 1.0.0

```
$ curl -sw '%{http_code} %{size_download}' -o /dev/null \
    localhost:8083/api/substrate/status
200 0
$ kubectl get actortemplates.ate.dev -A
error: the server doesn't have a resource type "actortemplates"
$ kubectl get sandboxagents -A
error: the server doesn't have a resource type "sandboxagents"

# REST is gone: kagent 1.0 speaks gRPC-Web, so my board got an empty 200.
# My board still asked for this CRD. It has ceased to be. It is an ex-CRD.
```

What to take home

- **Watch:** router parking and scheduler no_capacity, not CPU
- **Scale on:** busy + waiting, one jump up, the window peak down
- **Size your workers:** unsized, one pod advertises the whole node (upstream #3479)
- **Expect:** APIs that move, tools that fail quietly, ghost actors, backends that can't keep up
- **Run it:** github.com/themsquared/substrate-day-demo

AGENT SUBSTRATE DAY

Thank you

Mike Moore

github.com/themsquared/substrate-day-demo

@themsquared • webofmike.com

